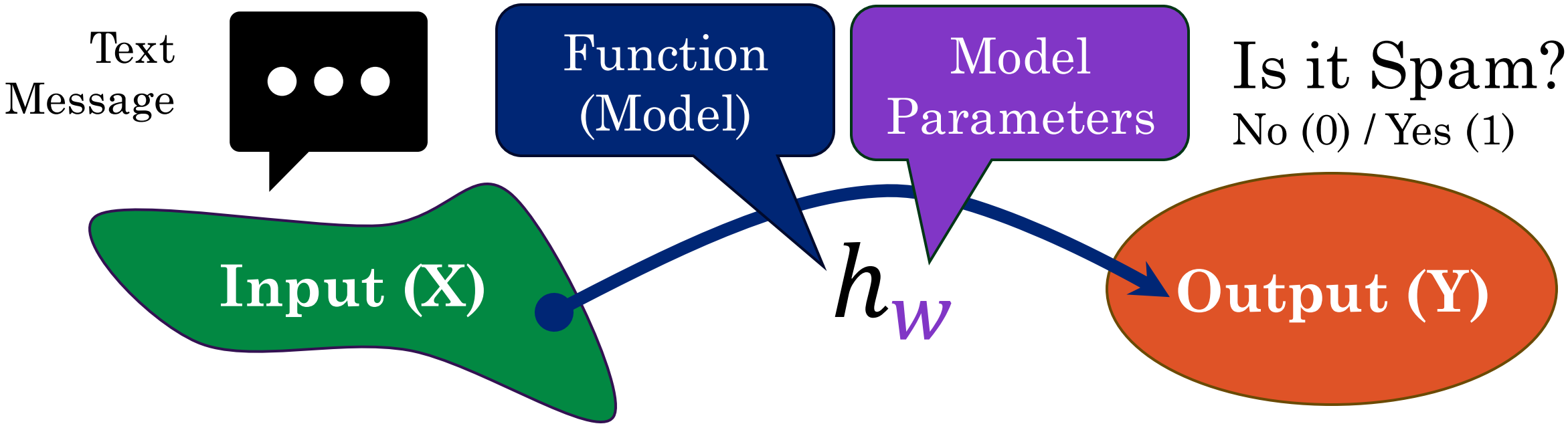
Introduction to ML Crash Course

22.11.2025

Furkan Cantürk

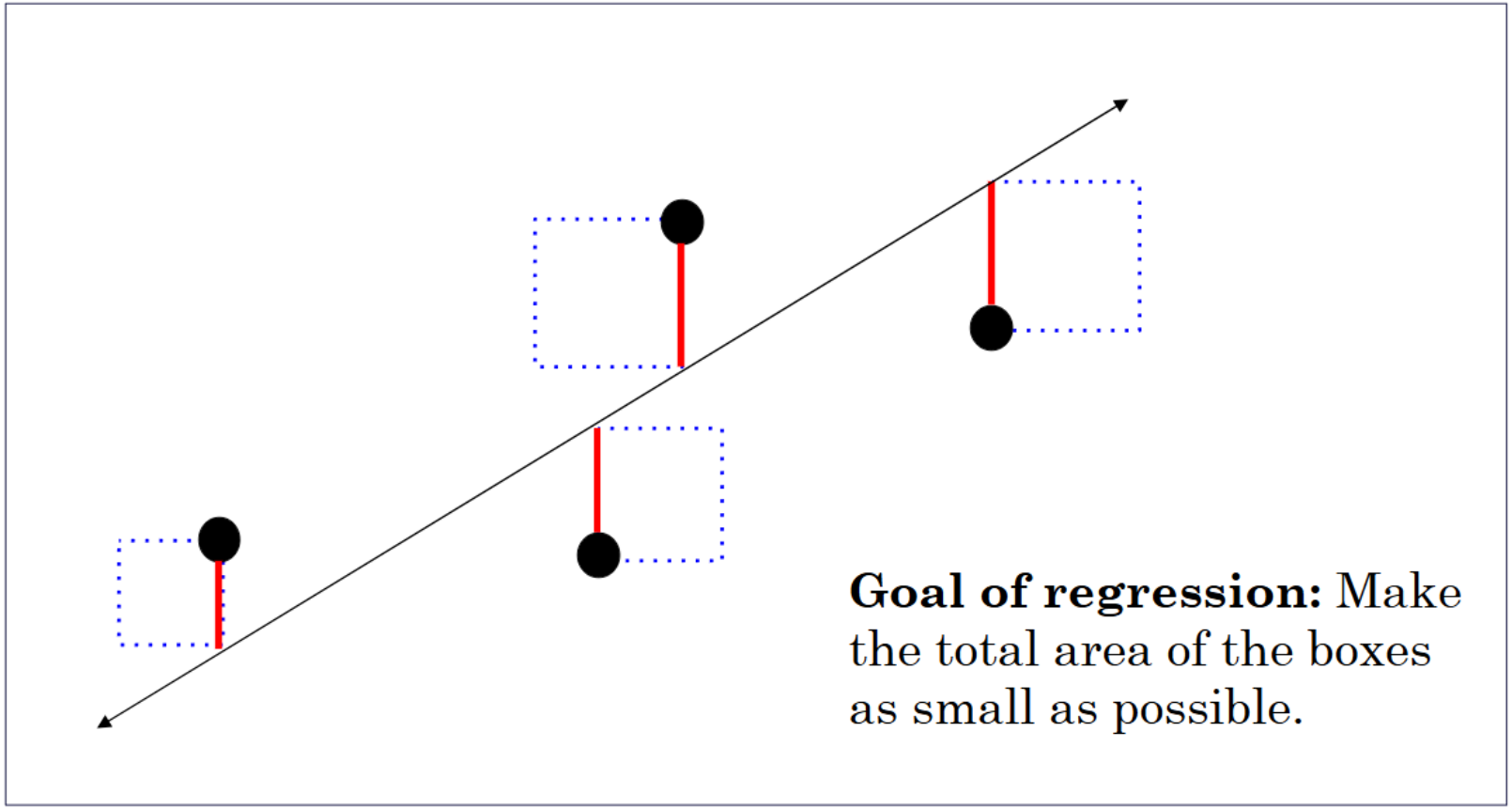
Machine Learning as **Learned Function Approximation**



Machine learning becomes the process of **"learning"** the **model parameters** from data or interaction with the world.

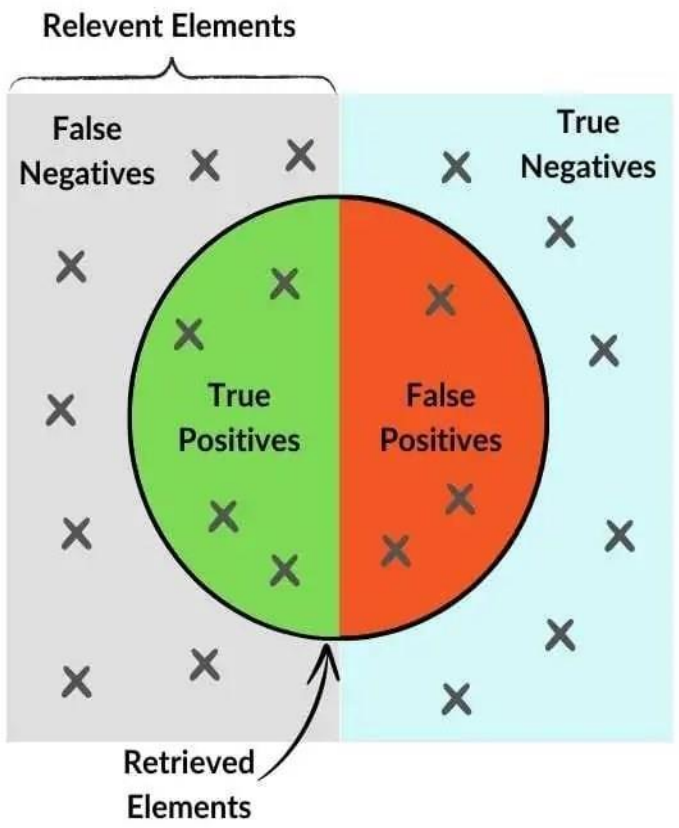
Slide take from CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025

Visualizing the Sum of Squared Error of Regression Model



Slide take from CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025

Precision & Recall



How many retrieved elements are relevant?

Precision = $\frac{\text{True Positives}}{\text{Retrieved Elements}}$

How many relevant elements are retrieved?

Recall = $\frac{\text{True Positives}}{\text{Relevent Elements}}$

Regression Evaluation - Metrics

Mean Squared Error (MSE)	$\frac{1}{N}\left(\sum_{n=1}^{N}\left(t_n - y(\mathrm{x}_n, w)\right)^2\right)$
Root Mean Squared Error (RMSE)	$\sqrt{\frac{1}{N}\left(\sum_{n=1}^{N}\left(t_n - y(\mathrm{x}_n, w)\right)^2\right)}$
R-Squared (R^2) Score	$1 - \frac{\sum_{n=1}^{N}\left(t_n - y(\mathrm{x}_n, w)\right)^2}{\sum_{n=1}^{N}(t_n - \bar{t_n})^2}$
Mean Absolute Error (MAE)	$\frac{1}{N}\left(\sum_{n=1}^{N}\lvert t_n - y(\mathrm{x}_n, w)\rvert\right)$
Mean Absolute Percentage Error (MAPE)	$\frac{1}{N}\left(\sum_{n=1}^{N}\frac{t_n - y(\mathrm{x}_n, w)}{t_n}\right) \times 100$

Slide take from [CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025]

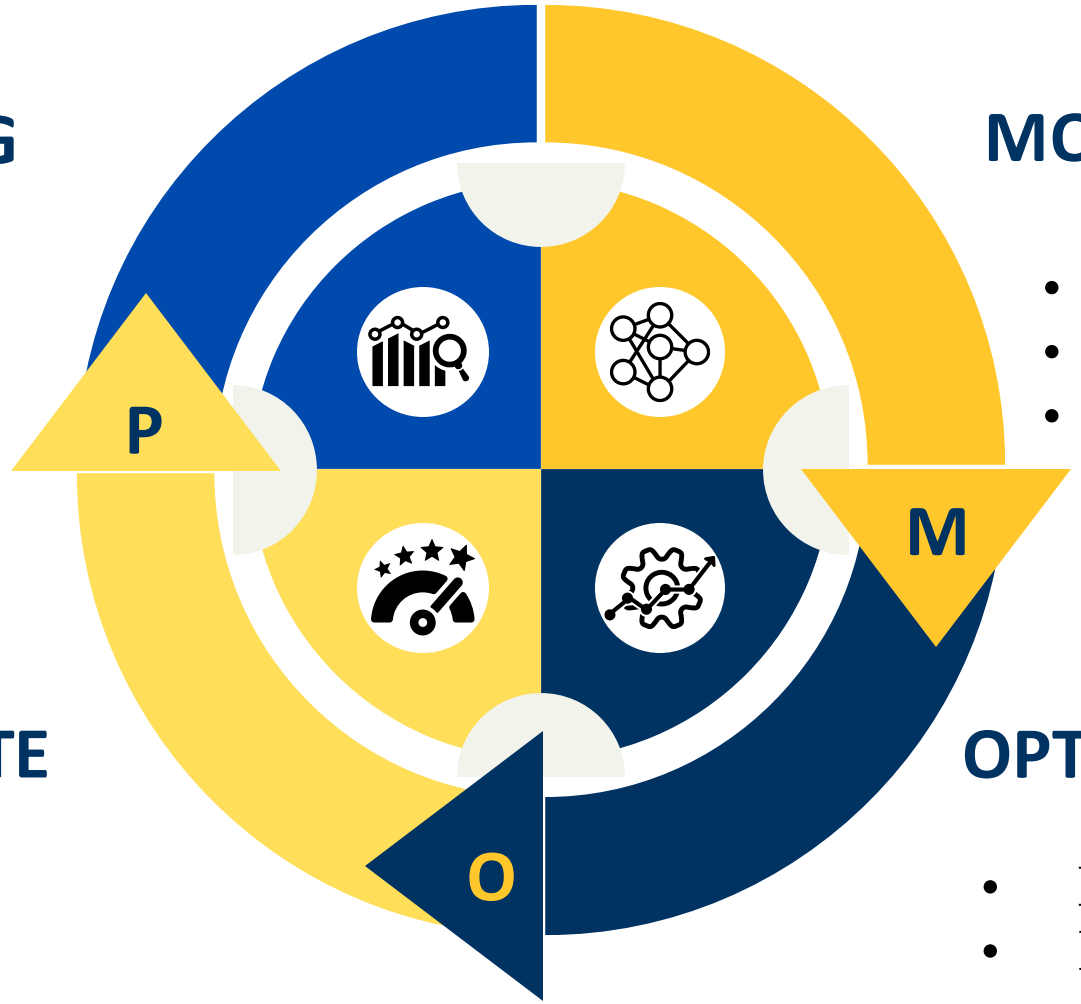
ML Lifecycle

LEARNING PROBLEM

- Target
- Objective
- Data

MODEL DESIGN

- Model family/Architecture
- Hypothesis space
- Inductive biases / Assumptions



PREDICT & EVALUATE

- Making predictions
- Accuracy Metrics

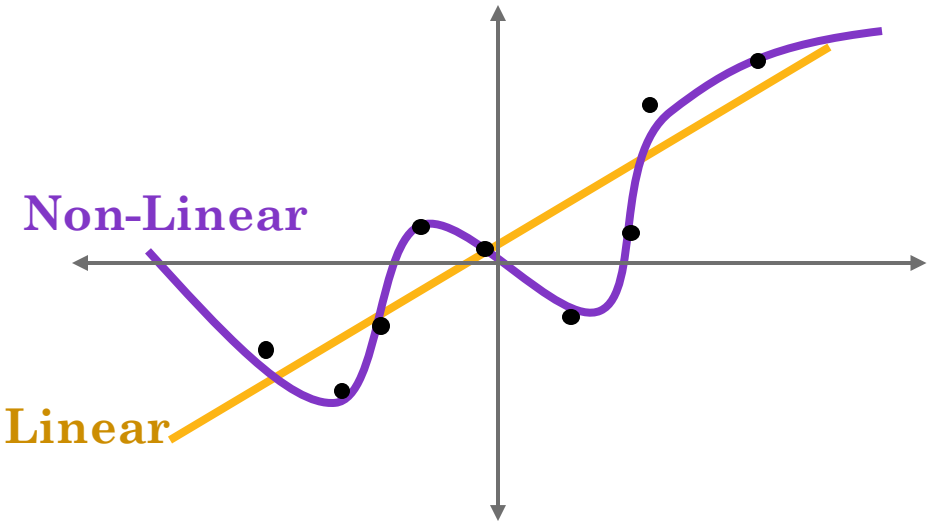
OPTIMIZATION

- Iterative Opt. Algorithms
- Hyperparameter Tuning

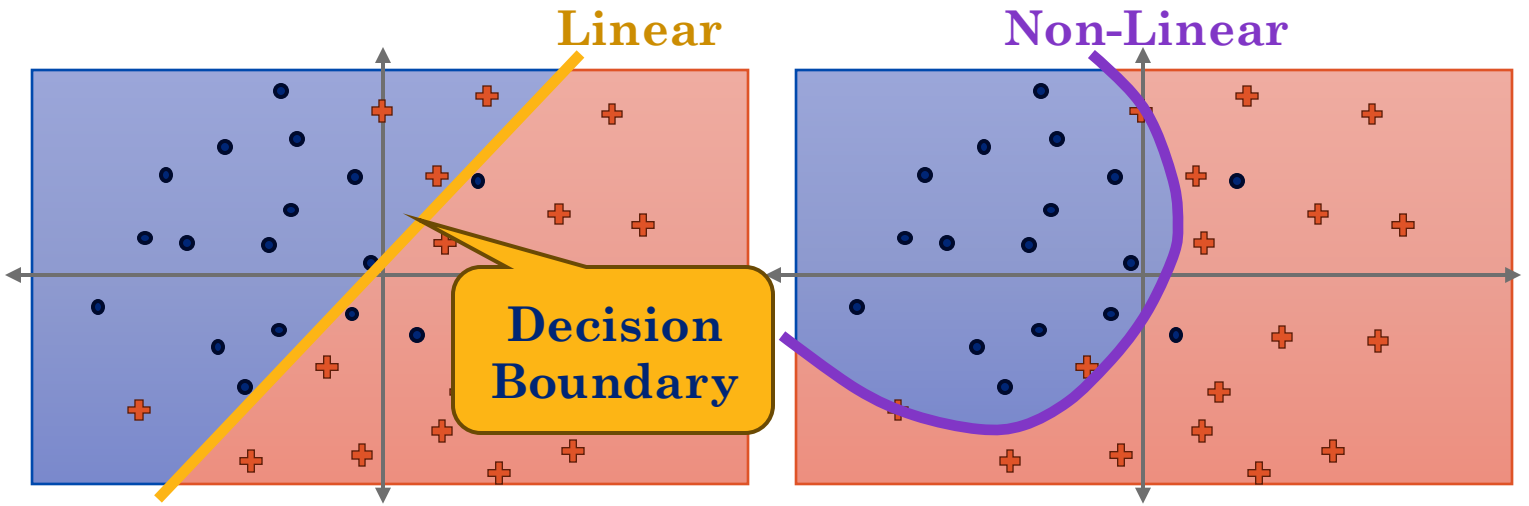
Slide take from CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025

Complexity:
Linear Models vs Non-Linear Models

Regression



Classification (Decision Boundary)

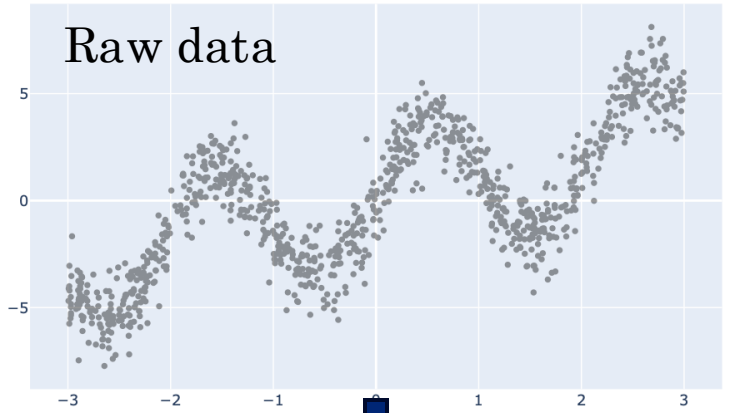


Nonlinear models are more expressive and can represent more complex relationships.

Are non-linear models better?

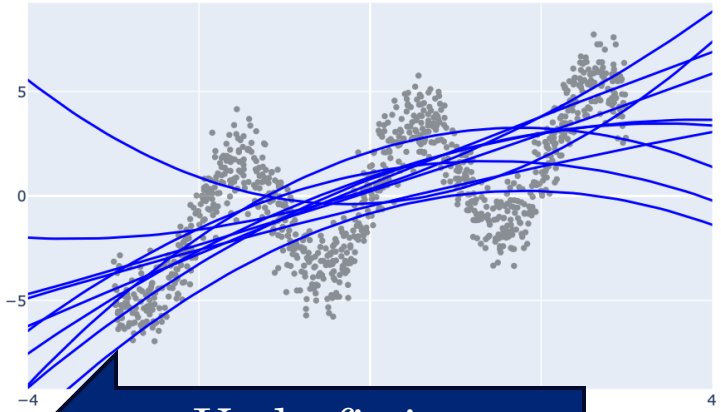
Slide take from CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025

Complexity and Overfitting

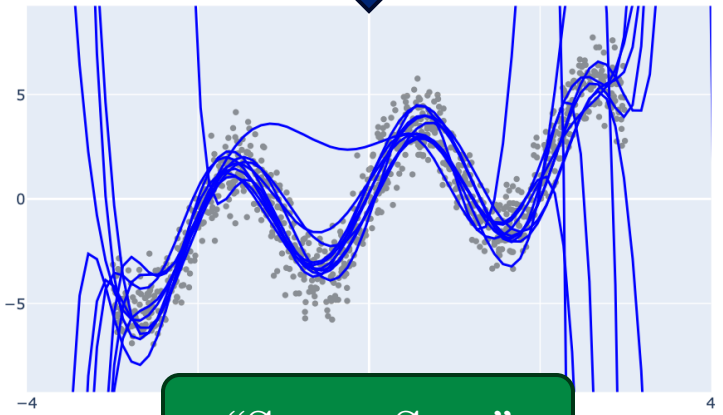


More complex isn't always better.

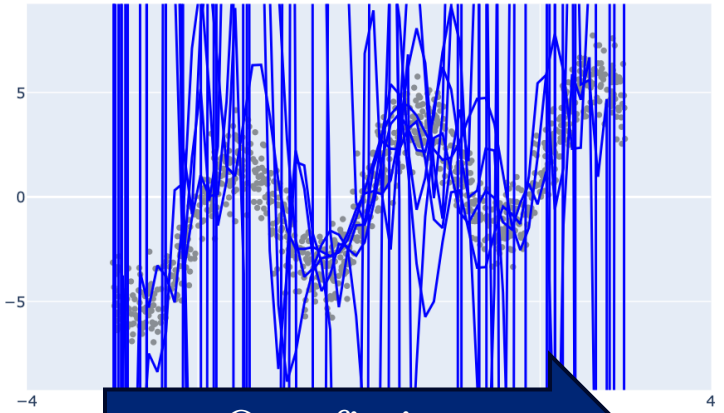
Fit models to samples of data



Underfitting



"Sweet Spot"



Overfitting

Slide take from [CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025]

Loss (Error) Curves

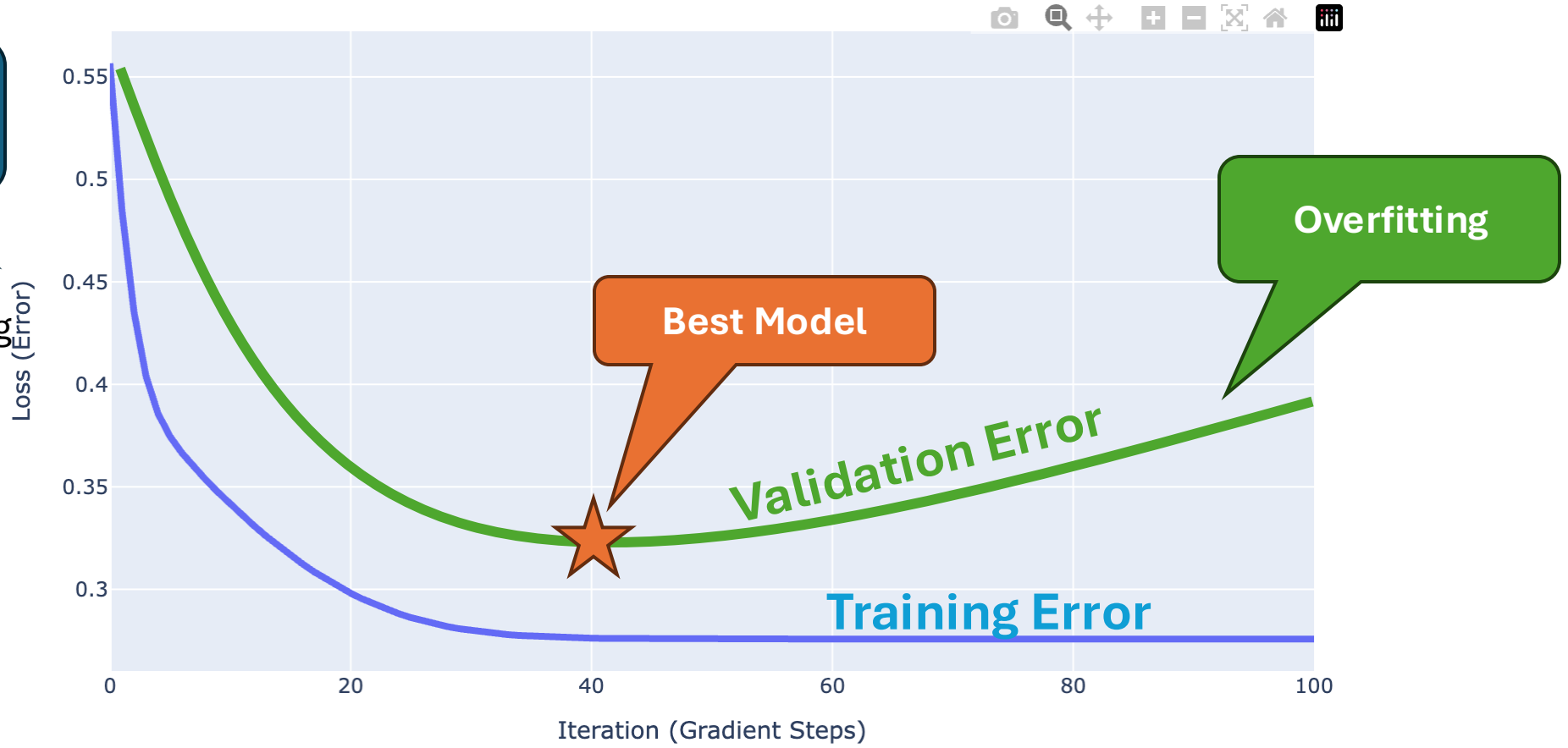
The **loss** or **error curve plots** the error as a function of the number of **iterations** (**steps**) of gradient descent.

Training and **Validation** Error

Validation curves are typically **above** the training curves.

You can **overoptimize** the objective (**overfit**).

Choose parameters with **lowest validation error**.

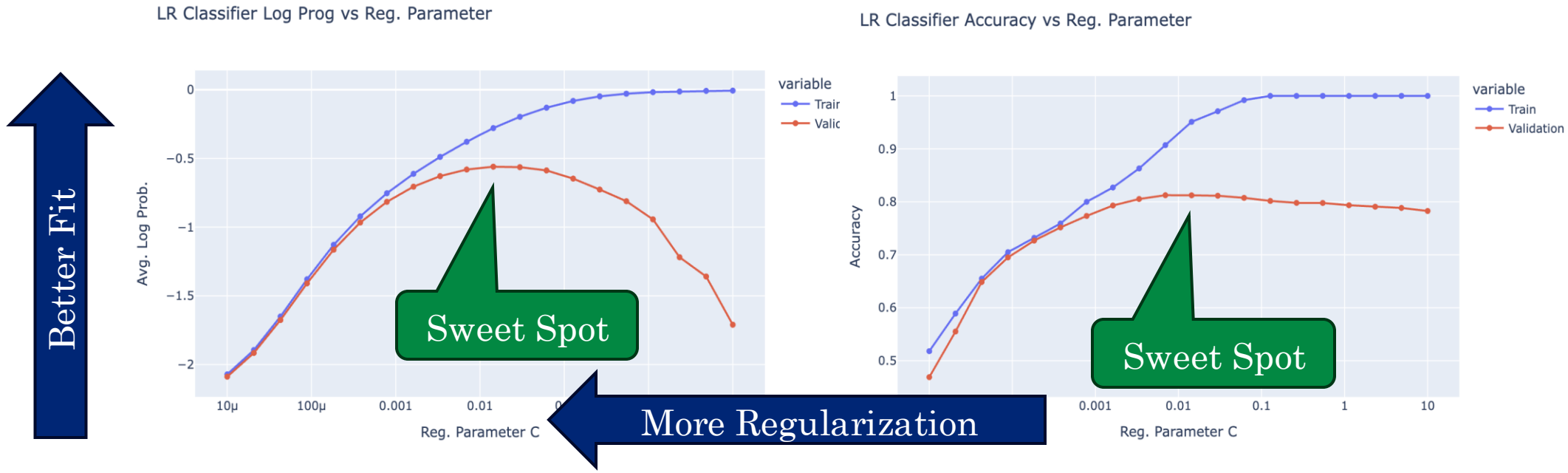


Slide take from CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025

Training vs Validation Accuracy

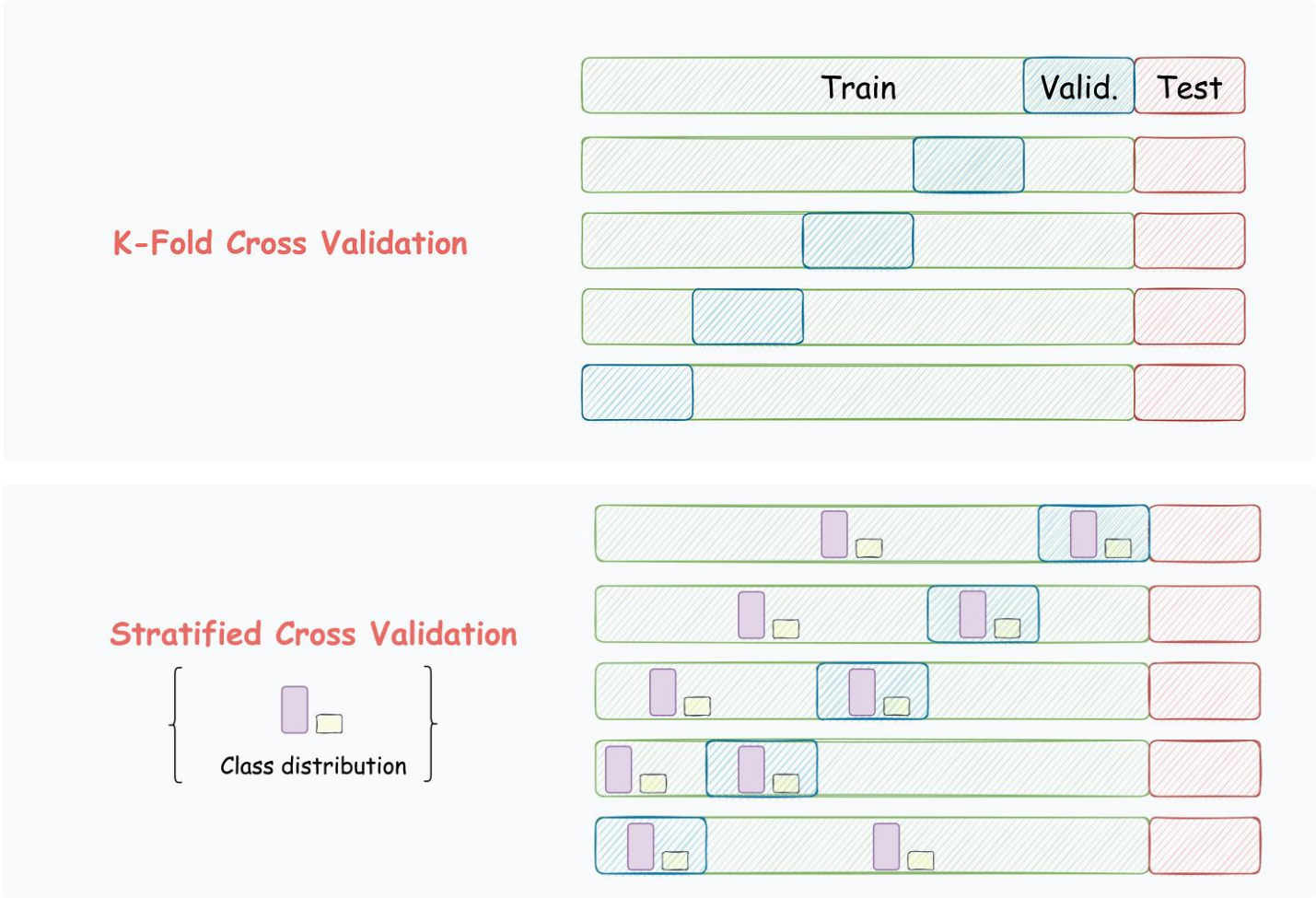
When tuning **regularization hyperparameters** against the validation dataset, it is common to see plots where

- training acc. **is greater than** validation accuracy
- acc. on the **training dataset continues to increase.**
- acc. on the **validation** dataset **increases** and **then decreases.**



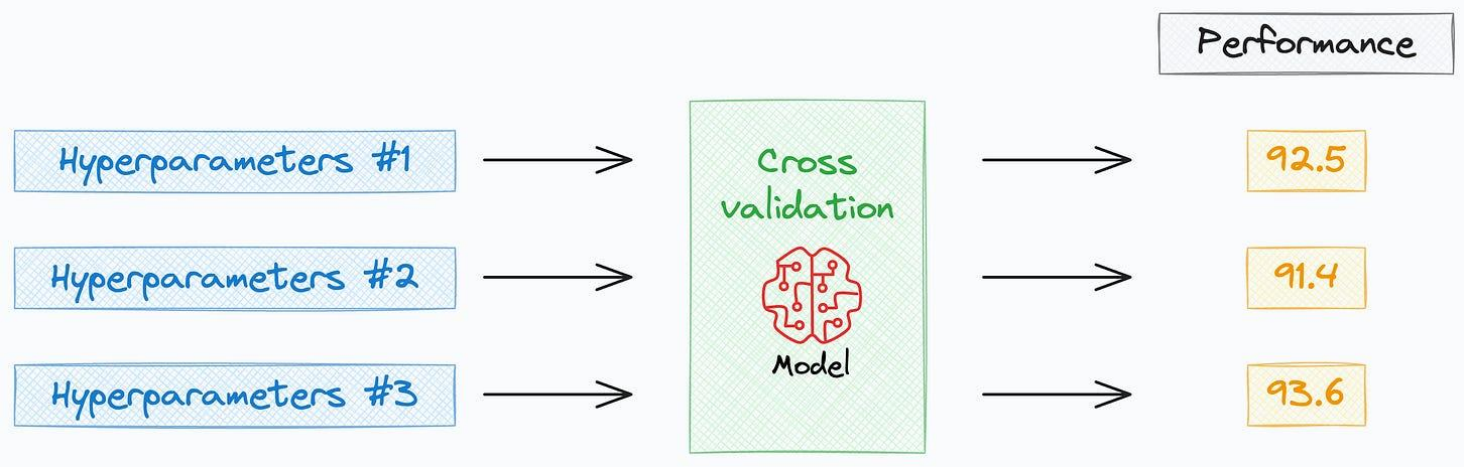
Slide take from [CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025]

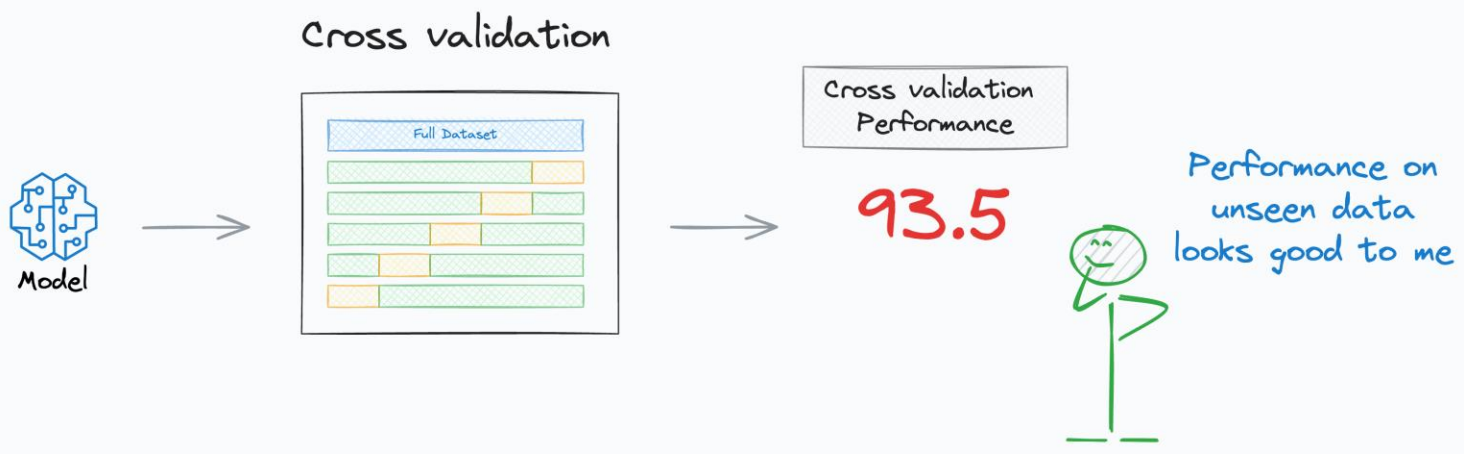
Cross Validation



Source: https://blog.dailydoseofds.com/p/5-cross-validation-techniques-explained

What to Do After Cross Validation?





Source: https://blog.dailydoseofds.com/p/you-cross-validated-the-model-what

Hyperparameters

The **hyperparameters** are the parameters that are constant during the optimization (training) algorithm.

- Example $\boldsymbol{\lambda}$ in $\hat{w} = \arg\min_{w \in \Theta} \textbf{Error}\left[h_w; \mathfrak{D}_{\text{training}}\right] + \boldsymbol{\lambda}\, \textbf{Reg}[w]$

The **hyperparameters** are chosen to **improve generalization** performance as **measured on** the **validation dataset**.

- Hyperparameters are often selected using a **grid-search**

```
for hp1 in [0.1, 1, 10]:
    for hp2 in [0.1, 1, 10]:
        model = MyModel(hp1=hp1, hp2=hp2)
        model.fit(X_train, y_train)
        error[hp1, hp2] = error(y_val, model.predict(X_val))
hp1_best, hp2_best = arg_min(error)
```

	0.1	1	10
0.1	.8	.7	.7
1	.4	.2	.3
10	.5	.6	.6

Slide take from CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025

Iterative Optimization Algorithms

We typically use **iterative optimization algorithms** to train (fit) the model by solving the following problem:

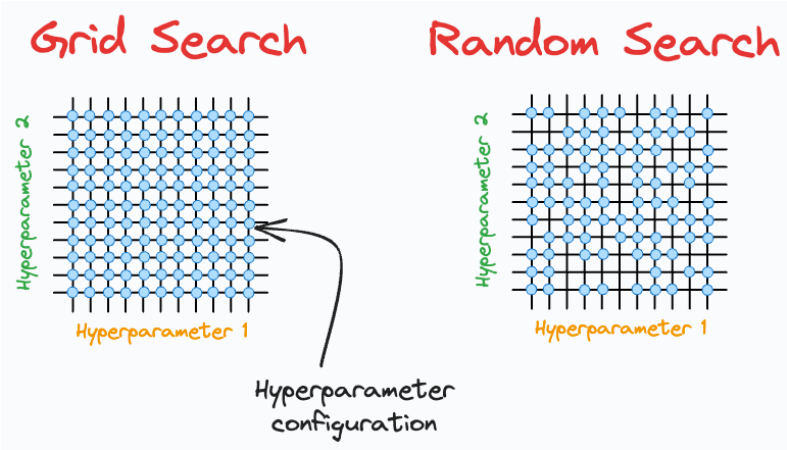
$$\hat{w} = \arg\min_{w \in \Theta} \textbf{Error}\left[h_w; \mathfrak{D}_{\text{training}}\right] + \boldsymbol{\lambda}\, \textbf{Reg}[w]$$

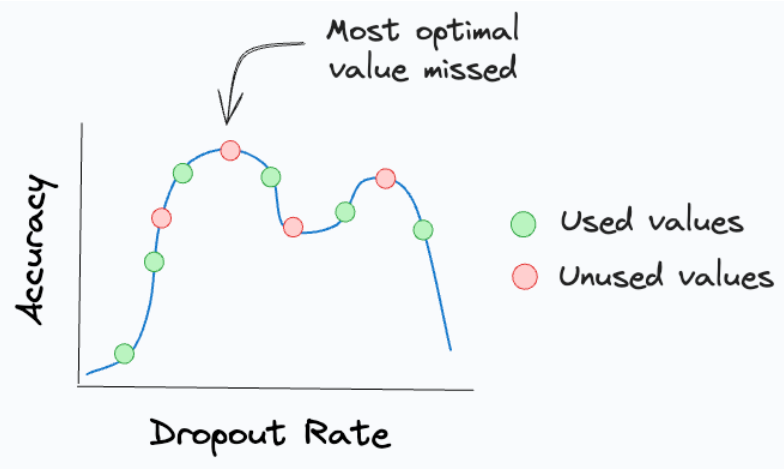
- These algorithms can be **slow** and **require tuning**
  - Often **approximate** and return **local minima**
- Tuning can affect **generalization performance**

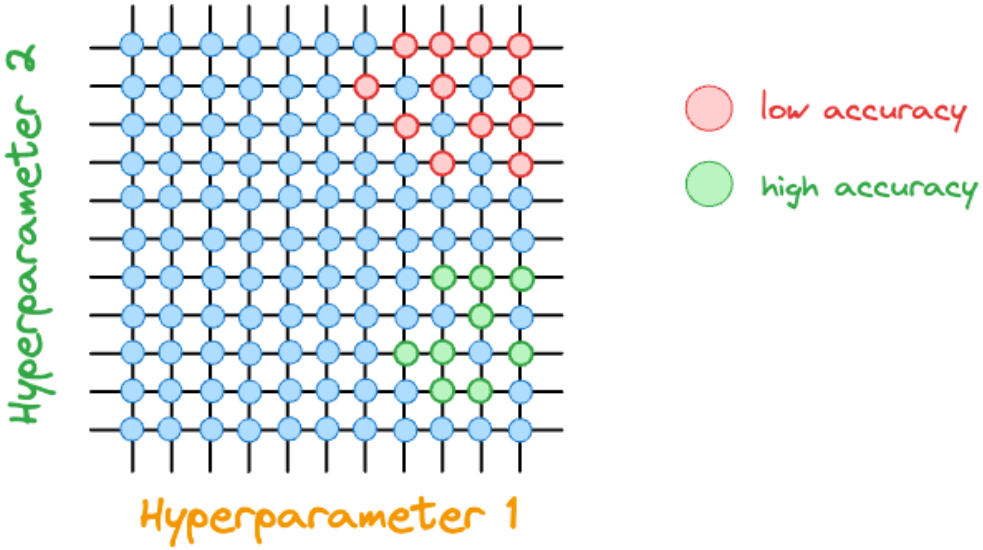
This slide will be the focus of a significant part of the class.

Slide take from CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025

Grid Search vs. Random Search vs. Bayesian Optimization







	Grid Search	Random Search	Bayesian Optimization	
Total Iterations	720	360	100	7x less iterations ✅
Total Run-time	242s	113s	48s	5x less run-time ✅
Best Trial Index	667	221	83	Optimal config. found earlier ✅
Best F1 score	0.93	0.93	0.93	Same score ✅

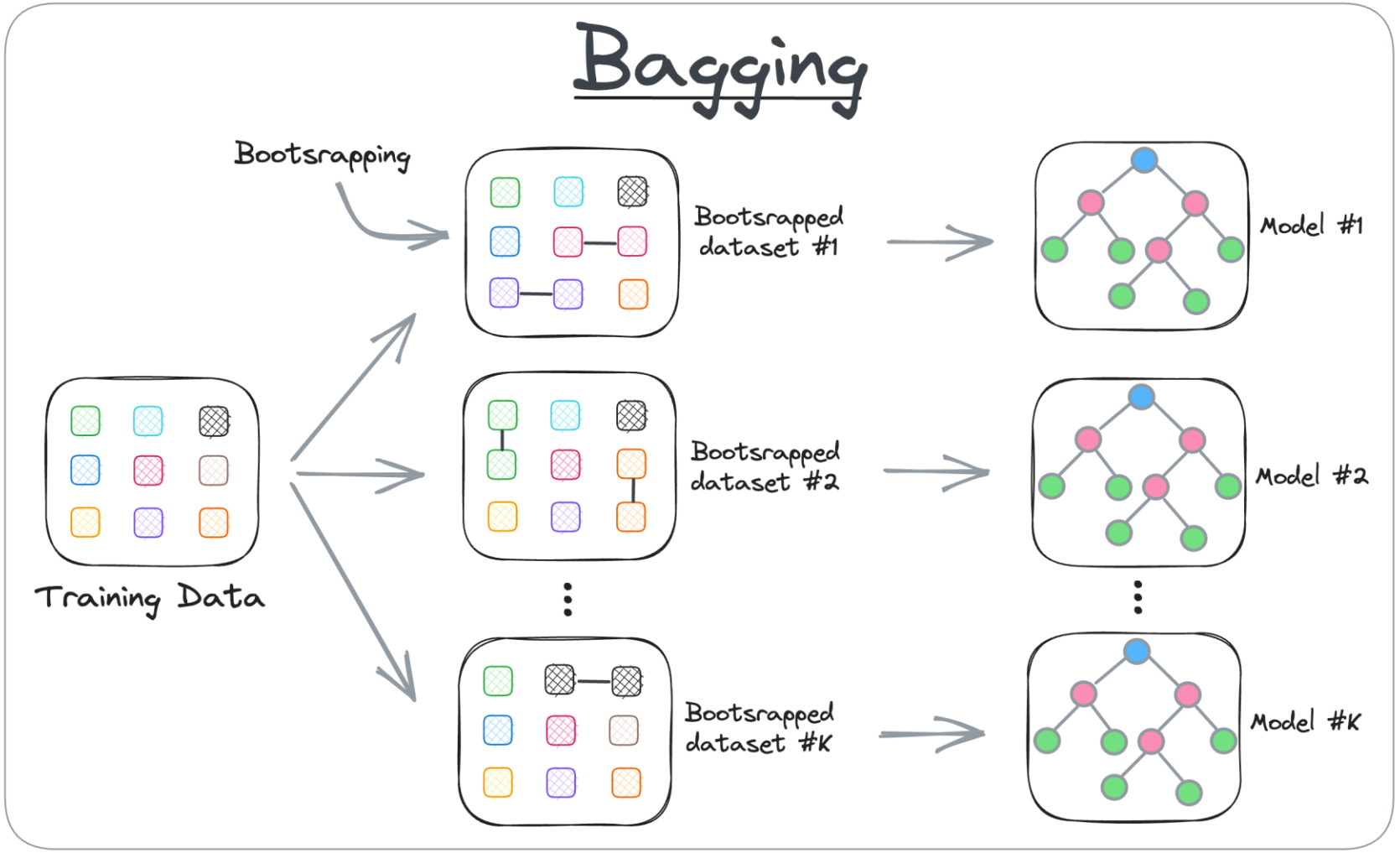
Sources: https://blog.dailydoseofds.com/p/grid-search-vs-random-search-vs-bayesian
https://www.dailydoseofds.com/bayesian-optimization-for-hyperparameter-tuning/

Bagging (Bootstrapped Aggregation)

- Bootstrapping creates different subsets of data
- Trains one model per subset
- Aggregates all predictions to get the final prediction

Some common Bagging models:

- Random Forests
- Extra Trees



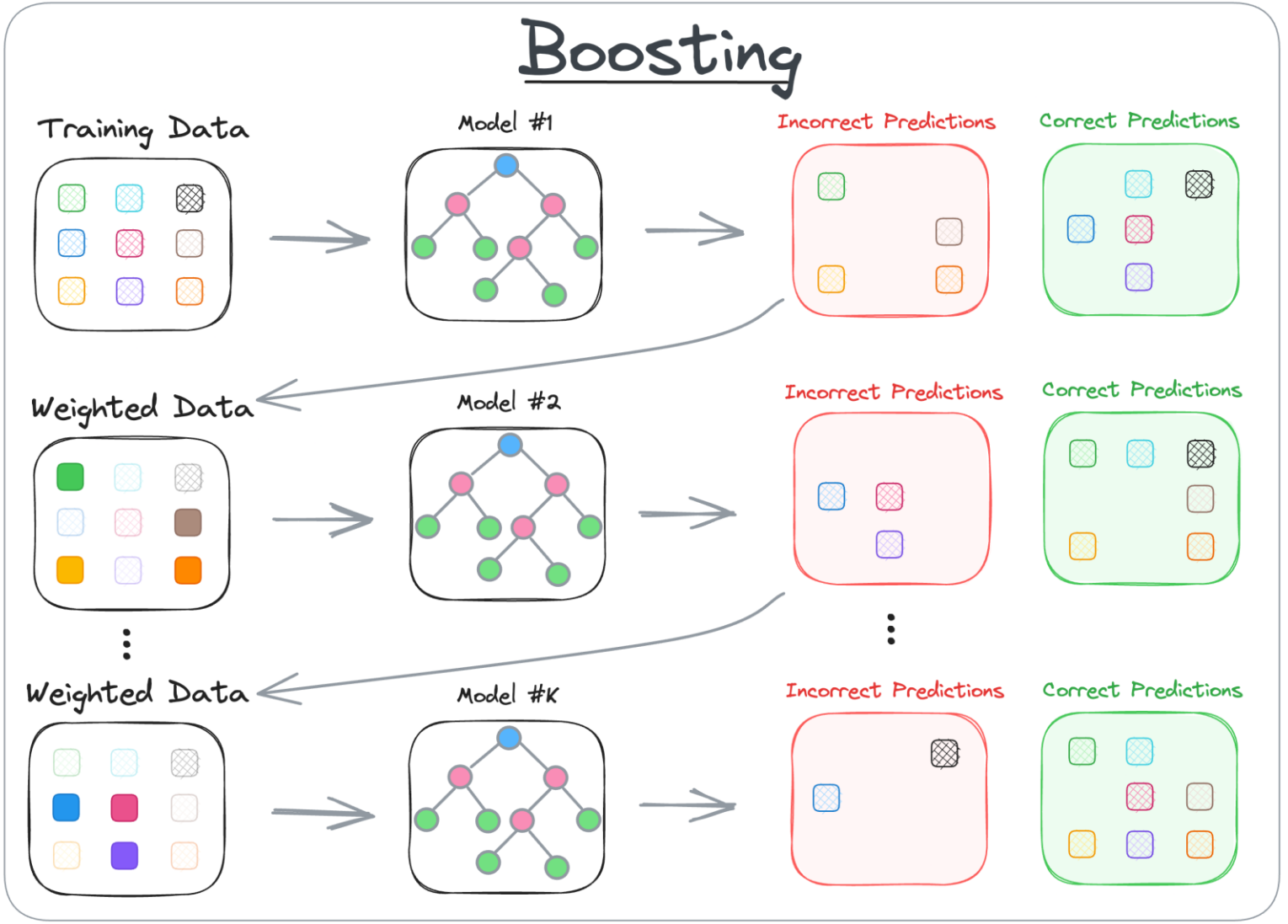
Source: https://blog.dailydoseofds.com/p/a-visual-and-overly-simplified-guide-4b1

Boosting

- An iterative training process
- The subsequent model puts more focus on misclassified samples from the previous model.
- The final prediction is a weighted combination of all predictions.

Some common Boosting models:

- XGBoost
- CatBoost
- LightGBM



Source: https://blog.dailydoseofds.com/p/a-visual-and-overly-simplified-guide-4b1

Further Reading on Gradient Boosting Machine Models

- eXtreme Gradient Boosting (XGBoost)
- Categorical Boosting (CatBoost)
- Light Gradient Boosting Machine (LightGBM)







- https://blog.dailydoseofds.com/p/top-gradient-boosting-methods

References

- CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025
- Daily-Dose-of-Data-Science/tree/main
- 48-most-popular-open-ml-datasets

Advised Learning & Tutorial Materials For Beginner Level Machine Learning

Python

- https://www.learnpython.org/
- https://www.py4e.com/lessons

Books

- Python Data Science Handbook
- Introduction to Machine Learning with Python
- CS 229 — Machine Learning Cheatsheet by Shervine Amidi

Courses

- CS 189/289A: Intro to Machine Learning, UC Berkeley, Fall 2025
- Deep Learning Course by Prof. Bryce

Blog

- https://blog.dailydoseofds.com/
- Daily-Dose-of-Data-Science/tree/main

Videos / Yotube Lists

- Deep Learning Codes and Concepts (Simply Explained)
  - Gradient Boosting Machines
  - Time series
  - Machine Learning by StatsQuest
  - Deep Learning by StatsQuest

Interactive Tools

- https://ml-playground.com/
- Gradient Descent with two parameters
- Tinker With a Neural Network